

Informe Técnico — Trabajo Práctico Integrador (Base De Datos)

Materia: Base De Datos

Comisión: N° 4

Profesor: Sergio Neira

Autores:

Nombre
Enzo Giaquinta
Ramiro Salcedo
Ignacio Sanchez
Fernando Torrez
Agustin Contardi

Proyecto: Base de datos food_store (PostgreSQL) para un comercio de alimentos, cargada con datos sintéticos a gran escala y optimizada mediante índices, vistas estándar y una vista materializada de facturación.

1) Elementos implementados en cada unidad

Carpeta food_store/ — Núcleo del Trabajo Práctico Integrador

Archivo	Elemento implementado
schema.sql	Esquema relacional completo: tipo enumerado forma_pago_enum, tablas cliente, categoria, producto, pedido, detalle_pedido, con claves primarias, claves foráneas, restricciones CHECK e índices de rendimiento base (idx_pedido_cliente, idx_producto_categoria_activo).

data.sql	Carga masiva de datos sintéticos: 10 categorías, 20.000 clientes, 50.000 productos, 200.000 pedidos y ~700.000 líneas de detalle_pedido (2 a 5 productos por pedido), con ANALYZE final.
queries.sql	Consultas analíticas del workload (3.1 Top 10 clientes por gasto, 3.2 Productos sobre promedio de categoría) con verificación de equivalencia EXCEPT bidireccional, y ejemplos de EXPLAIN comentados.
views.sql	Tres vistas estándar: vista_productos_vigentes_categoria, vista_pedidos_usuario (oculta email/teléfono por menor privilegio) y vista_detalle_pedido_producto (con subtotal), cada una con su verificación EXCEPT.
indices.sql	Cuatro índices de optimización: idx_cliente_activo (parcial), idx_detalle_pedido_id_pedido, idx_producto_categoria_precio (compuesto y parcial), idx_pedido_id_cliente.
informe_mediciones.md	Documento con todas las mediciones de rendimiento, planes de ejecución y pruebas de impacto en escrituras.
specs/	Especificaciones técnicas por feature (vistas e índices).
DUIA.md	Bitácora de uso de IA (Kiro y OpenCode) con las decisiones aceptadas y descartadas.

Además, en la **Parte C** documentada en DUIA.md se implementó la **vista materializada de facturación mensual** mv_facturacion_categoria_mes con su índice único (categoria_id, mes) para permitir refresco concurrente.

Carpeta TP2/ — Concurrencia y lectura crítica

- **informe_concurrencia.md:** reproducción y análisis de tres escenarios de concurrencia sobre la tabla de prueba test:
 - **Lectura no repetible** (Read Committed vs REPEATABLE READ).
 - **Lectura fantasma** (Read Committed vs REPEATABLE READ).
 - **Espera por bloqueo** (SELECT ... FOR UPDATE y alternativas NOWAIT / SKIP LOCKED).

- **ejercicio_lectura_critica.md**: análisis línea por línea y corrección de dos scripts SQL con errores:
 - UPDATE funcion SET activa = FALSE **sin WHERE** (desactivaba todas las funciones, no solo las de películas retiradas de cartel) → corregido con WHERE pelicula_id IN (SELECT ... WHERE retirada_de_cartel = TRUE).
 - DELETE FROM categoria WHERE id NOT IN (...) — comportamiento incorrecto ante NULL en categoria_id → corregido con NOT EXISTS.

Carpeta TP3/ — Competencia de optimización y baseline

- **competencia_query_base.sql**: consulta base sin índices (listado de productos por categoría con precio_lista BETWEEN 1500 AND 2000).
- **competencia_equipo_a.sql**: estrategia A — índice B-tree simple y parcial idx_producto_precio_simple sobre precio_lista.
- **competencia_equipo_b.sql**: estrategia B — índice covering idx_producto_precio_covering con INCLUDE (id_categoria, nombre, stock) para lograr Index Only Scan.
- **explain_analyze_baseline.sql**: medición de la consulta original (pedidos con más de 3 productos y monto > \$5000) y de dos búsquedas puntuales nuevas sobre detalle_pedido (por id_pedido y por id_producto), antes y después de crear índices dedicados.
- **specs.md**: especificación de consultas (SPEC 1: Top 5 categorías por ingreso; SPEC 2: productos nunca vendidos).
- **carga_datos.sql / carga_detalle_pedido.sql**: scripts de generación de datos sintéticos.

Carpeta TP4/ — Reglas de negocio y consultas

- **migrations_rules.sql**: migración idempotente de reglas de negocio:
 - Constraint CHECK de formato de email en cliente (Regla 1).
 - Trigger trg_valida_categoria_activa: un producto solo puede pertenecer a una categoría **activa** (Regla 2).
 - Trigger trg_valida_categoria_desactivacion: no se puede desactivar una categoría que tenga productos activos asociados (Regla 3).
- **test_rules.sql**: script de pruebas de las reglas (fallo esperado con ROLLBACK y éxito con COMMIT).
- **consulta_3_1_top_clientes_gasto.sql** y **consulta_3_2_precio_sup_promedio_categoria.sql**: dos versiones de cada consulta (generada con IA y alternativa propia con CTE / funciones de ventana) más verificación de equivalencia con EXCEPT bidireccional.

2) Cómo probaron el funcionamiento

Carga de datos

1. Se creó la base local **tp_food_store** (puerto 5433 por defecto en las conexiones).
2. Se ejecutó `food_store/schema.sql` para crear el esquema.
3. Se ejecutó `food_store/data.sql` para la carga masiva (todo dentro de transacciones, con ANALYZE final).
4. Verificación con conteos: `SELECT COUNT(*)` sobre producto, pedido y detalle_pedido.

Vistas estándar

- Se ejecutó `food_store/views.sql`.
- Para cada una de las 3 vistas se corrió la verificación de equivalencia **EXCEPT bidireccional** contra la consulta manual equivalente: ambos EXCEPT (vista vs manual y manual vs vista) deben devolver **0 filas**.

Índices y consultas

- **Baseline:** antes de crear índices se ejecutaron las consultas con EXPLAIN (ANALYZE, BUFFERS, TIMING) capturando plan, tiempos y buffers.
- Se ejecutó `food_store/indices.sql` (idempotente gracias a `CREATE INDEX IF NOT EXISTS`).
- Se re-ejecutaron las mismas consultas con el mismo EXPLAIN y se compararon los resultados. Para asegurar baselines puros se realizó `DROP INDEX` antes de cada *profiling* (aislamiento de pruebas por consulta).
- **Impacto en escrituras:** se probó con cargas masivas tipo `INSERT ... SELECT` antes y después de crear los índices (100.000 filas en detalle_pedido, 10.000 en producto y 20.000 en pedido), comparando el Execution Time y el tiempo de los triggers de validación de clave foránea.

Competencia TP3

- Se ejecutó la **misma consulta base** en tres configuraciones: sin índices adicionales, con el índice de Equipo A y con el índice covering de Equipo B, siempre con EXPLAIN (ANALYZE, BUFFERS, TIMING).

Concurrencia TP2

- Con **dos sesiones** (psql/DBeaver) sobre la base foodstore_test:
 - Sesión 1 abre transacción y lee; Sesión 2 escribe y hace COMMIT; Sesión 1 vuelve a leer (se detecta lectura no repetible / fantasma).
 - Se repitió con ISOLATION LEVEL REPEATABLE READ para verificar que el problema desaparece.
 - Escenario de bloqueo: Sesión 1 con SELECT ... FOR UPDATE, Sesión 2 intenta lo mismo y queda en espera hasta el COMMIT.

Reglas de negocio TP4

- Se ejecutó test_rules.sql: cada prueba envuelta en BEGIN ... ROLLBACK cuando debe **fallar** (la excepción aborta la transacción) y en BEGIN ... COMMIT cuando debe **pasar**.

Vista materializada (Parte C)

- Se contrastó SELECT * FROM mv_facturacion_categoria_mes contra la consulta analítica sin materializar, verificando que los totales coinciden exactamente.
- Se validó el refresco con REFRESH MATERIALIZED VIEW CONCURRENTLY, posible gracias al índice único (categoria_id, mes), y que no bloquea las lecturas en uso.

3) Resultados obtenidos

Carga de datos

Elemento	Resultado
Categorías	10
Clientes	20.000
Productos	50.000
Pedidos	200.000
Líneas de detalle_pedido	~699.423

Vistas estándar

- Los **6 bloques EXCEPT** (3 vistas × 2 direcciones) devolvieron **0 filas**, confirmando la equivalencia de cada vista contra su consulta manual.

Concurrencia (TP2)

Escenario	Read Committed	REPEATABLE READ
Lectura no repetible	El monto cambió de 100.00 a 999.99 (problema reproducido)	Se mantuvo en 100.00 (resuelto)
Lectura fantasma	El COUNT pasó de 3 a 4 (problema reproducido)	Se mantuvo en 3 (resuelto)
Bloqueo FOR UPDATE	Sesión 2 quedó bloqueada hasta el COMMIT de Sesión 1	Alternativas: NOWAIT (error inmediato) o SKIP LOCKED (omite filas bloqueadas)

Reglas de negocio (TP4)

Prueba	Regla	Resultado esperado	Resultado
1. Email inválido	Regla 1 (CHECK)	Falla	✓ Falla
2. Email válido	Regla 1 (CHECK)	Pasa	✓ Pasa
3. Producto con categoría inactiva	Regla 2 (trigger)	Falla	✓ Falla
4. Producto con categoría activa	Regla 2 (trigger)	Pasa	✓ Pasa
5. Desactivar categoría con productos activos	Regla 3 (trigger)	Falla	✓ Falla
6. Desactivar categoría sin productos activos	Regla 3 (trigger)	Pasa	✓ Pasa

Vista materializada (Parte C)

- Los totales de facturación y unidades de mv_facturacion_categoria_mes coinciden de forma **exacta** con la consulta analítica sin materializar.
- El refresco CONCURRENTLY funciona sin bloquear lecturas (requisito del índice único).

Resumen de tiempos (tabla)

Consulta / Prueba	Antes	Después	Mejora
Competencia base (BETWEEN 1500-2000)	10.866 ms	—	baseline
Competencia Equipo A (índice simple parcial)	—	6.001 ms	~45%
Competencia Equipo B (índice covering)	—	2.327 ms	~79% vs base
3.1 Top 10 clientes por gasto	3612.970 ms	3478.948 ms	~134 ms
3.2 Precio superior al promedio de categoría	251402.102 ms	149374.371 ms	~102 s
3.3 Pedidos > 3 productos y monto > \$5000	4107.925 ms	3521.245 ms	~586 ms
Búsqueda por id_pedido en detalle_pedido	0.205 ms	0.198 ms	~3%
Búsqueda por id_producto en detalle_pedido	57.378 ms	0.157 ms	~99,7%
Escritura: INSERT 100.000 en detalle_pedido	9994.689 ms	9672.286 ms	~3% (marginal)

Escritura: INSERT 10.000 en producto	709.408 ms	356.046 ms	sin penalidad medible
Escritura: INSERT 20.000 en pedido	1014.174 ms	1348.252 ms	+33% (overhead del B- tree ~281 ms $\approx$ 14 μ s/fila)

4) Consultas optimizadas y diferencias antes y después

Consulta base de competencia (TP3)

- **Antes (sin índice):** Seq Scan sobre producto + Hash Join contra categoria, filtrando en el heap todas las filas activas fuera del rango (Rows Removed by Filter: 45045). Tiempo: **10.866 ms**.
- **Equipo A (índice simple parcial precio_lista):** Bitmap Index Scan + Bitmap Heap Scan que limita el rango de precios. Tiempo: **6.001 ms**.
- **Equipo B (índice covering con INCLUDE):** Index Only Scan Backward (sin tocar el heap, Heap Fetches: 0) + Memoize sobre categoria. Tiempo: **2.327 ms**.

Diferencias clave: la estrategia covering reemplaza el Seq Scan por un Index Only Scan y evita re-leer categoria con un Memoize, reduciendo los buffers accedidos y eliminando el ordenamiento costoso sobre el heap parcial.

Consultas analíticas del informe de mediciones

3.1 — Top 10 clientes por gasto

- **Antes:** 3612.970 ms, con Seq Scan sobre detalle_pedido, pedido y cliente, y ordenamiento externo en disco (external merge Disk: 34568kB). Planning: 9.482 ms.
- **Después:** 3478.948 ms, misma estrategia general (el optimizador mantiene Seq Scan porque la consulta agrega toda la tabla antes del LIMIT 10), pero con menor planning (0.494 ms). La mejora es acotada (~134 ms) porque el cuello de botella es el escaneo y agregación completos, no el filtrado.

3.2 — Productos con precio superior al promedio de su categoría

- **Antes (peor caso):** 251402.102 ms (~4,19 minutos). Las subconsultas correlacionadas se re-evaluaban 44.935 veces (SubPlan 2) y 22.425 veces

(SubPlan 1) con Bitmap Heap Scan, forzando **35.099.763** *shared hits* (35 GB de lecturas en buffers). Costo estimado: 33.493.893,77.

- **Después:** 149374.371 ms (~2,5 minutos). El índice compuesto y parcial `idx_producto_categoria_precio` permitió transformar las subconsultas a **Index Only Scan con Heap Fetches: 0**, reduciendo los *shared hits* a **1.298.390** (más del **96% menos** de lecturas) y el costo estimado a 8.540.012,90 (~75% menos de costo de plan).

3.3 — Pedidos con más de 3 productos y monto total > \$5000

- **Antes:** 4107.925 ms, planning 23.549 ms. El Merge Join usaba la PK compuesta `pk_detalle_pedido` (costo del nodo: 61.028,05).
- **Después:** 3521.245 ms, planning 3.760 ms. El Merge Join usa el índice dedicado `idx_detalle_pedido_id_pedido` (costo del nodo: 45.411,16), independizando el plan de acceso de la estructura de la PK compuesta.

Búsquedas puntuales (TP3 / `explain_analyze_baseline.sql`)

- **Por `id_pedido = 1`:** Bitmap Heap Scan usando la PK compuesta (0.205 ms) → Bitmap Heap Scan usando `idx_detalle_pedido_id_pedido` (0.198 ms).
- **Por `id_producto = 2` (caso radical):** como `id_producto` es la segunda columna de la PK compuesta, no se podía usar directamente → **Parallel Seq Scan sobre ~700.000 filas en 57.378 ms**. Con `idx_detalle_pedido_id_producto` pasa a un **Index Scan directo en 0.157 ms** (mejora de ~99,7%).

Prueba de escrituras (INSERT)

- **detalle_pedido: costo marginal (~3%)** — el tiempo está dominado por la validación de las FKs `fk_detalle_pedido_pedido` y `fk_detalle_pedido_producto` (~6 s del total); la actualización del B-tree sobre `id_pedido` es despreciable.
- **producto: sin penalidad medible** — la corrida con índice resultó incluso más rápida (709.408 → 356.046 ms) por el estado del buffer cache; la caída del trigger `fk_producto_categoria` (214.166 → 60.235 ms) confirma que la diferencia es ruido y no costo de mantenimiento del índice.
- **pedido: overhead observable (+33%)** — insertar 20.000 entradas nuevas en el B-tree de `id_cliente` (`idx_pedido_id_cliente`) suma ~334 ms; descontando el ruido del trigger `fk_pedido_cliente` (+9%), el costo neto del índice es ~281 ms (~14 µs por fila), acotado y amortizable frente a la aceleración que aporta en consultas.

Conclusión de la optimización: los índices permitieron cambiar planes costosos (Seq Scan, Bitmap Heap Scan y Parallel Seq Scan) por accesos dirigidos (Index Only Scan

con Heap Fetches: 0, Memoize, Index Scan), con reducciones de tiempo de hasta el 99,7% y de I/O de más del 96% en el caso más crítico (3.2). El costo de mantenimiento en escrituras resultó marginal salvo en el caso de pedido, donde sigue siendo acotado.

5) Otras herramientas de IA utilizadas (además de las dadas por la cátedra)

No se utilizó ninguna herramienta de IA fuera de las indicadas por la cátedra.

Las herramientas utilizadas fueron:

Herramienta	Tipo	Finalidad
Kiro	IA dada por la cátedra	Especificación de las vistas, índices y consultas (specs/).
OpenCode	IA dada por la cátedra	Agente de codificación en terminal: generación y edición de los scripts SQL (views.sql, indices.sql, queries.sql, vista materializada) a partir de los specs.
psql / DBeaver	Clientes	Ejecución de scripts, medición y profiling con EXPLAIN (ANALYZE, BUFFERS, TIMING).

Decisiones aceptadas

- **Índices de SPEC-001, SPEC-002 y SPEC-003:** se aceptaron las propuestas de índices (idx_cliente_activo, idx_detalle_pedido_id_pedido, idx_producto_categoria_precio, idx_pedido_id_cliente) incorporando IF NOT EXISTS para garantizar idempotencia.
- **Vista vista_pedidos_usuario:** se aceptó ocultar email y telefono (principio de menor privilegio), aunque la propuesta inicial los exponía.
- **Vista vista_detalle_pedido_producto:** se aceptó agregar la columna subtotal y **rechazar el filtro por producto.activo**, porque las líneas ya facturadas no deben desaparecer si el producto se dio de baja después (dato histórico).
- **Vista materializada:** se aceptó el script con WITH DATA (la pobla desde el inicio) y el **índice único (categoria_id, mes)**, condición necesaria para REFRESH MATERIALIZED VIEW CONCURRENTLY sin bloquear lecturas.

- **Competencia TP3:** se validó que la estrategia **covering (Equipo B)** supera a la simple (Equipo A) en la consulta de rango de precios.

Decisiones descartadas

- **Índice B-tree completo sobre producto(activo):** descartado por baja cardinalidad (columna booleana con ~95% en TRUE); no aporta selectividad.
- **Índice simple sobre producto(precio_lista) sin id_categoria:** descartado porque las subconsultas de la 3.2 agrupan por id_categoria; sin la clave de correlación no permite localizar los precios por categoría.
- **Índices sobre cantidad o precio_unitario_facturado en detalle_pedido:** descartados porque el filtro es posterior al agrupamiento (HAVING), donde un índice B-tree no aporta selectividad previa.
- **Índice compuesto sobre pedido(fecha_pedido, forma_pago):** descartado porque ambas columnas solo participan en proyección y agrupamiento, no como predicados de filtrado.
- **Índices simples sobre columnas de agregación (cantidad, precio_unitario_facturado):** descartados por no aportar selectividad previa al GROUP BY y encarecer las escrituras.
- **Exponer email y telefono en vista_pedidos_usuario:** descartado por criterio de seguridad.
- **Filtrar por producto.activo en vista_detalle_pedido_producto:** descartado por ser dato histórico facturado.